Information Technology University



Lab Manual

for

CS220: Artificial Intelligence

Course Instructor : **Husnain Haider**
Semester Fall 2023 : **3**
Teaching Assistant : **Muhammad Ahmad**
Teaching Assistant : **Moeed Ahmad**
Teaching Assistant : **Muzamil Ali**
Teaching Assistant : **Abdul Rehman Ameen**
Teaching Assistant : **Muhammad Anas**

Student Name :
Student Roll No. :

Department of Computer Science ITU, Lahore, Pakistan

ADVERSARIAL SEARCHES

Task 1: Optimal Leaf Node Search with Minimax Algorithm

Objective: Implement the Minimax algorithm to find the optimal leaf node in a generated tree. In this task, students should generate a binary tree up to a specified depth (e.g., depth 3 or 4), with terminal nodes assigned random utility values. Students must traverse the tree using the Minimax algorithm to identify the leaf node that represents the optimal outcome for the maximizing player.

Instructions:

1. Generate a binary tree where each terminal node has a randomly assigned utility value.
2. Implement the Minimax algorithm, alternating between maximizing and minimizing levels.
3. Trace the recursive calls to identify the path and the chosen leaf node with the optimal outcome.
4. Output the selected leaf node, along with the path and calculated values at each node.

Task 2: Optimal Leaf Node Search with Alpha-Beta Pruning

Objective: Use the Alpha-Beta Pruning optimization to improve the efficiency of searching for the optimal leaf node in the tree. Students will generate a similar tree structure as in Task 1 but use Alpha-Beta Pruning to avoid unnecessary branches.

Instructions:

1. Use the same binary tree generation process as in Task 1.
2. Implement Alpha-Beta Pruning alongside the Minimax algorithm to ignore branches that do not influence the final decision.
3. Compare the number of nodes evaluated with and without Alpha-Beta Pruning.
4. Output the selected leaf node, the path, and a summary of pruned nodes.

Helpful Code:

```
# 3-Layer Tree Structure
root = TreeNode(children=[
    TreeNode(children=[TreeNode(value=3), TreeNode(value=5),
TreeNode(value=6)]),
    TreeNode(children=[TreeNode(value=9), TreeNode(value=1),
TreeNode(value=2)])
])
```

Output Code:

```
# Reset iteration counter and call the minimax function
minimax_iterations = 0
optimal_value = minimax(root, 3, True)
print("Optimal value found using Minimax:", optimal_value)
print("Total nodes evaluated in Minimax:", minimax_iterations)
```

Task-1

Example code:

```
# Define a tree node class for ease of use
class TreeNode:
# Global counter for iterations
# Minimax function with iteration counter
def minimax(node, depth, is_maximizing):
        # Base case
        if is_maximizing:
        else:
```

Task-2

Example code:

```
class TreeNode:
# Global counter for iterations
# Alpha-Beta Pruning function with iteration counter
def alphabeta(node, depth, alpha, beta, is_maximizing):
      # Base case
      # Beta cut-off
      # Alpha cut-off
```